ECE 3218

Lab #2 Report

Multiple Precision Arithmetic

Daniel Xu

September 17, 2026

Overview

The ATmega328P is an 8-bit microcontroller, so arithmetic operations on values wider than that must be built. In this lab, AVR assembly subroutines were written and tested to perform 32-bit addition, subtraction, and 32x32-bit multiplication.

Procedure

1. Data Layout and copybytes

Spaces for the operands and the sum were reserved in the data segment. The test operands were written into program memory with .db and copied into the SRAM at runtime to enable faster testing. The numbers were stored MSB first (at the lowest address).

The copybytes subroutine uses Z to point to the source in program memory and X to the destination in SRAM. r16 is the byte counter and the loop decrements r16 until it reaches 0. We use lpm to load byte from the program memory to r0 and st to store it into SRAM.

2. Passing arguments on the stack

Before calling add32bit/sub32bit, he program loads X with the address of value1 and pushes 8 bytes from the SRAM to the stack. Because rcall pushes the 2-byte return address, we pop it in the subroutine and store it into r18 and r19. So that we can pop the bytes of value1 and value2. We store these into a32v1.

3. 32-bit Addition

The two numbers are added one byte at a time, starting at the LSB. We use add on the LSB and then adc on the higher bytes so the carry out of each byte is accounted for in the next. We save the flags in SREG before returning because the loop to push the result would clear the flags. We then restore the flags right before executing ret.

4. 32-bit Subtraction

The subtraction subroutine reuses the addition routine, except we use two's complement to negate the second operand before adding.

5. 8x8-bit Multiplication

As per the lab instructions, I wrote an unsigned 8x8-bit multiply. This subroutine uses the concept of shifting and adding the first multiplicand. We also shift the second multiplicand right to check the next least significant bit. If it is a 0, no operation is done and we shift both multiplicands again. If it is a 1, we add the first (shifted at its current position) to the current result.

6. 32x32-bit Multiplication

We use three 2-byte pointers to perform this operation. We have the address of each of the operands and the product. The multiplication subroutine pops the return address and pointers into registers and clears the 8 product bytes. We perform the byte-by-byte multiplication for each combination of bytes (like you would do by hand in long multiplication). We store the partial product in r1 and r0, which is then added to the product and the carry propagates.

Signed multiplication uses a XOR on the two operands' sign bits and the result is the sign of the product. We perform a two's complement on any negative operand and perform multiplication normally, then do it again if the sign of the result is negative.

Results

By running the code on the Arduino with the Atmel-ICE like in Lab 1, we were able to see into the registers and SRAM of the board as the program ran.

I recorded the diagram of the stack at various points during a subroutine call of add32bit.

Address	1	2	3	4	5	6	7	8
0x8FF		0A	0A		0A	0A	0A	
0x8FE		0B	0B		0B	0B	0B	
0x8FD		0F	0F		11	11	11	
0x8FC		EF	EF		10	10	10	
0x8FB		00	00			10		
0x8FA		00	00			00		
0x8F9		01	01					
0x8F8		11	11					
0x8F7			10					
0x8F6			00					
SP	0x8FF	0x8F7	0x8F5	0x8FF	0x8FB	0x8F9	0x8FB	0x8FF

Key:

1	Before the parameter push
2	Before the subroutine call
3	After the subroutine call
4	After the subroutine pulls the parameters
5	After the subroutine pushes the result
6	Before the ret
7	After the ret
8	After the main retrieves the result

I observed the following behavior:

Test	A	B	A+B
1	0x0A0B0FEF	0x00000111	0x0A0B1100
2	0x000000FF	0x00000001	0x00000100

Test	A	B	A-B
1	0x0A0B0FEF	0x00000111	0x0A0B0EDE
2	0x00000003	0x00000005	0xFFFFFFFE

Test	A	B	A*B
1	0x0D	0x0B	0x008F
2	0x00	0xFF	0x0000

Test	A	B	A*B
1	0x00000FFF	0x00000111	0x110EEF
2	0x00000000	0xFFFFFFFF	0x000000…

All results came back as expected. I chose these numbers to test a normal case and edge cases like multiplying by 0, subtracting to get a negative number.

Post-Lab

What are the benefits of a simulation vs a 'real' AVR? What are the disadvantages?

The benefit of a simulation is that you can see how your program works without having physical access to a board. It also gives complete visibility into the processor and all of its registers. However, it does not reproduce real-world timing and clock issues, and you cannot connect external sensors and read from them in a simulator (you would have to make your own fake data).

In your programming class you learned about pass-by-value vs. pass-by-reference for subroutine arguments. Which method did you use in the first two programs? What about the third? Comment on the relative complexity of both methods.

The addition and subtraction routines are called by value. The main program pushes copies of the operand bytes, and the result comes back on the stack. Add32bit uses its own copy of the operands and does not touch the caller's variables. The multiplication routine uses a reference to memory locations. Pass-by-value is simpler to think about and protects data but requires more

usage of the stack. Pass-by-reference is more complicated to conceptualize but pushes a fixed number of addresses no matter how large the numbers are.

How is the stack initialized on the Arduino board? How is it initialized in the simulator? Where is the 'system,' stack on the simulator?

The stack is initialized by setting SPH and SPL to high(RAMEND) and low(RAMEND). The system stack is located at 0x8FF, just like on the real board and is accessible through the memory panel in Microchip Studio.

Discussion/Conclusion

This lab implemented multi-byte addition, subtraction and multiplication on an 8-bit processor by chaining byte operations through the carry flag and passing arguments onto the stack. All test cases matched their predicted results.

The most challenging part of the lab was managing the stack. Because we run the addition in a subroutine, we must balance push/pop operations to ensure we return to the correct place after we finish the subroutine. By having the stack pointer and stack contents visible during debugging in Microchip Studio, I was able to keep track of the state much easier.

Appendix: Code

32-bit Addition/Subtraction

```
    .cseg
    jmp main
main:
    ; copy 32 bit vals from flash into sram
    ldi zh, high(pmv1*2)
    ldi zl, low(pmv1*2)
    ldi xh, high(value1)
    ldi xl, low(value1)
    ldi r16, 8
    rcall copybytes

    ldi xh, high(value1) ; [x] <- addr(1st value MSB)
    ldi xl, low(value1);
    ldi r16, 8 ; countdown for two 4-byte numbers
```

```
pushloop:
    ld r0, x+ ; get values 1 and 2 from data memory
    push r0 ; push them to stack
    dec r16 ; MSB is first in
    brne pushloop ; continue till 8 bytes are done
    rcall sub32bit ; call the add function and return here when we are
done

fin:
    rjmp fin

add32bit:
    ; save return address
    pop r19 ; LSB of ra
    pop r18 ; MSB of ra
    ; get numbers off stack and into local mem
    ldi xh, high(a32v1+8)
    ldi xl, low(a32v1+8)
    ldi r16,8 ; byte countdown

poploop:
    pop r0 ; pop byte off stack
    st -x, r0 ; store it in data memory
    dec r16 ; and continue until 8 bytes are done
    brne poploop

    ; adding each byte
    ; add LSB
    lds r20, a32v1+3
    lds r21, a32v1+7
    add r20, r21 ; doesnt read carry flag but writes it for the next op
    sts a32sum+3, r20

    ; next bytes
    lds r20, a32v1+2
    lds r21, a32v1+6
    adc r20, r21
    sts a32sum+2, r20

    lds r20, a32v1+1
    lds r21, a32v1+5
```

```
    adc r20, r21
    sts a32sum+1, r20

    ; add MSB
    lds r20, a32v1
    lds r21, a32v1+4
    adc r20, r21

    in r4, SREG ; preserve sreg
    sts a32sum, r20
    ldi xh, high(a32sum) ; [x] <- addr(sum MSB)
    ldi xl, low(a32sum)
    ldi r16, 4 ; number of bytes to push

pushloop2:
    ; push the result onto the stack
    ld r0, x+ ; get sum byte from data mem
    push r0 ; push to stack
    dec r16 ; MSB is first in
    brne pushloop2 ; continue until 4 bytes are done
    out sreg, r4 ; restore status register
    push r18 ; MSB of return address
    push r19 ; LSB of return address
    ret

sub32bit:
    ; save return address
    pop r19
    pop r18
    ; takes two's complement of subtrahent
    ; first flip the bits
    lds r20, value2
    com r20
    sts value2, r20

    lds r20, value2+1
    com r20
    sts value2+1, r20

    lds r20, value2+2
    com r20
```

```
    sts value2+2, r20

    lds r20, value2+3
    com r20
    sts value2+3, r20

    ; addds 1 to complete the two's complement

    ldi r20, 1

    lds r21, value2+3
    add r21, r20
    sts value2+3, r21

    ldi r20, 0

    lds r21, value2+2
    adc r21, r20
    sts value2+2, r21

    lds r21, value2+1
    adc r21, r20
    sts value2+1, r21

    lds r21, value2
    adc r21, r20
    sts value2, r21

    ldi xh, high(value1)
    ldi xl, low(value1)
    ldi r16, 8

subpushloop:
    ld r0, x+
    push r0
    dec r16
    brne subpushloop
    push r18
    push r19
    rjmp add32bit
```

```
copybytes:
    lpm r0, z+ ; load byte form program memory, increment z
    st x+, r0 ; store byte into sram, increment x
    dec r16 ; decrement byte counter
    brne copybytes
    ret

pmv1: ; hard coded data stored in program memory
    .db 0x0A, 0x0B, 0x0F, 0xEF
    .db 0x00, 0x00, 0x01, 0x11

    .dseg
value1: .byte 4
value2: .byte 4
a32v1: .byte 8
a32sum: .byte 4
```

32-bit Multiplication

```
.cseg
    jmp main


main:
    ; copy two 32-bit values from FLASH into SRAM
    ldi zh, high(pmv1*2)
    ldi zl, low(pmv1*2)

    ldi xh, high(value1)
    ldi xl, low(value1)

    ldi r16, 8
    rcall copybytes

    ; multiply value1 * value2
    rcall mul32bit


fin:
    rjmp fin

mul32bit:
```

```
    ; clear 64-bit product
    ldi xh, high(product)
    ldi xl, low(product)
    ldi r16, 8
    clr r20

mulclr:
    st x+, r20 ; set all of result sram to 00
    dec r16
    brne mulclr


    ; byte 3 * byte 3, MSB first
    lds r20, value1+3
    lds r21, value2+3
    mul r20, r21            ; r1:r0 is the product (it is 16 bits)

    lds r22, product+7 ; LSB
    add r22, r0        ; r0 has LSB of product. no carry yet
    sts product+7, r22

    lds r22, product+6 ; second LSB
    adc r22, r1        ; r1 has MSB of product. no carry yet
    sts product+6, r22


    ; byte 3 * byte 2
    lds r20, value1+3
    lds r21, value2+2
    mul r20, r21

    lds r22, product+6
    add r22, r0
    sts product+6, r22

    lds r22, product+5
    adc r22, r1
    sts product+5, r22

    ldi r23, 0
```

```
    lds r22, product+4
    adc r22, r23
    sts product+4, r22


    ; byte 3 * byte 1
    lds r20, value1+3
    lds r21, value2+1
    mul r20, r21

    lds r22, product+5
    add r22, r0
    sts product+5, r22

    lds r22, product+4
    adc r22, r1
    sts product+4, r22

    ldi r23, 0
    lds r22, product+3
    adc r22, r23
    sts product+3, r22


    ; byte 3 * byte 0
    lds r20, value1+3
    lds r21, value2
    mul r20, r21

    lds r22, product+4
    add r22, r0
    sts product+4, r22

    lds r22, product+3
    adc r22, r1
    sts product+3, r22

    ldi r23, 0
    lds r22, product+2
    adc r22, r23
    sts product+2, r22
```

```
; byte 2 * byte 3
lds r20, value1+2
lds r21, value2+3
mul r20, r21

lds r22, product+6
add r22, r0
sts product+6, r22

lds r22, product+5
adc r22, r1
sts product+5, r22

ldi r23, 0
lds r22, product+4
adc r22, r23
sts product+4, r22


; byte 2 * byte 2
lds r20, value1+2
lds r21, value2+2
mul r20, r21

lds r22, product+5
add r22, r0
sts product+5, r22

lds r22, product+4
adc r22, r1
sts product+4, r22

ldi r23, 0
lds r22, product+3
adc r22, r23
sts product+3, r22


; byte 2 * byte 1
```

```
    lds r20, value1+2
    lds r21, value2+1
    mul r20, r21

    lds r22, product+4
    add r22, r0
    sts product+4, r22

    lds r22, product+3
    adc r22, r1
    sts product+3, r22

    ldi r23, 0
    lds r22, product+2
    adc r22, r23
    sts product+2, r22


    ; byte 2 * byte 0
    lds r20, value1+2
    lds r21, value2
    mul r20, r21

    lds r22, product+3
    add r22, r0
    sts product+3, r22

    lds r22, product+2
    adc r22, r1
    sts product+2, r22

    ldi r23, 0
    lds r22, product+1
    adc r22, r23
    sts product+1, r22


    ; byte 1 * byte 3
    lds r20, value1+1
    lds r21, value2+3
    mul r20, r21
```

```
lds r22, product+5
add r22, r0
sts product+5, r22

lds r22, product+4
adc r22, r1
sts product+4, r22

ldi r23, 0
lds r22, product+3
adc r22, r23
sts product+3, r22


; byte 1 * byte 2
lds r20, value1+1
lds r21, value2+2
mul r20, r21

lds r22, product+4
add r22, r0
sts product+4, r22

lds r22, product+3
adc r22, r1
sts product+3, r22

ldi r23, 0
lds r22, product+2
adc r22, r23
sts product+2, r22


; byte 1 * byte 1
lds r20, value1+1
lds r21, value2+1
mul r20, r21

lds r22, product+3
add r22, r0
```

```
    sts product+3, r22

    lds r22, product+2
    adc r22, r1
    sts product+2, r22

    ldi r23, 0
    lds r22, product+1
    adc r22, r23
    sts product+1, r22


    ; byte 1 * byte 0
    lds r20, value1+1
    lds r21, value2
    mul r20, r21

    lds r22, product+2
    add r22, r0
    sts product+2, r22

    lds r22, product+1
    adc r22, r1
    sts product+1, r22

    ldi r23, 0
    lds r22, product
    adc r22, r23
    sts product, r22


    ; byte 0 * byte 3
    lds r20, value1
    lds r21, value2+3
    mul r20, r21

    lds r22, product+4
    add r22, r0
    sts product+4, r22

    lds r22, product+3
```

```
    adc r22, r1
    sts product+3, r22

    ldi r23, 0
    lds r22, product+2
    adc r22, r23
    sts product+2, r22


    ; byte 0 * byte 2
    lds r20, value1
    lds r21, value2+2
    mul r20, r21

    lds r22, product+3
    add r22, r0
    sts product+3, r22

    lds r22, product+2
    adc r22, r1
    sts product+2, r22

    ldi r23, 0
    lds r22, product+1
    adc r22, r23
    sts product+1, r22


    ; byte 0 * byte 1
    lds r20, value1
    lds r21, value2+1
    mul r20, r21

    lds r22, product+2
    add r22, r0
    sts product+2, r22

    lds r22, product+1
    adc r22, r1
    sts product+1, r22
```

```
    ldi r23, 0
    lds r22, product
    adc r22, r23
    sts product, r22


    ; byte 0 * byte 0
    lds r20, value1
    lds r21, value2
    mul r20, r21

    lds r22, product+1
    add r22, r0
    sts product+1, r22

    lds r22, product
    adc r22, r1
    sts product, r22


    ; MUL writes to r1:r0
    ; restore r1 to zero
    clr r1

    ret


copybytes:
    lpm r0, z+
    st x+, r0
    dec r16
    brne copybytes
    ret

pmv1:
    .db 0x00, 0x00, 0x0F, 0xFF
    .db 0x00, 0x00, 0x01, 0x11


    .dseg

value1:.byte 4
```

```
value2:.byte 4
product:.byte 8
```

1-byte Multiplication

```
    .cseg
    .org 0x0000
    jmp main

main:
    ldi r16, high(RAMEND)   ; stack for rcall
    out sph, r16
    ldi r16, low(RAMEND)
    out spl, r16

    ldi zh, high(pmv1*2)    ; flash -> value1, value2
    ldi zl, low(pmv1*2)
    ldi xh, high(value1)
    ldi xl, low(value1)
    ldi r16, 2
    rcall copybytes

    rcall mult1x1
    rjmp fin

mult1x1:
    lds r20, value1     ; multiplicand, low half of r21:r20
    clr r21
    lds r17, value2     ; multiplier
    clr r18         ; product low
    clr r19         ; product high
    ldi r22, 8      ; bit counter

m_loop:
    ; checking if the bit is 0 or 1
    sbrs r17, 0     ; if 1: skip the rjmp, we just add the shifted
multiplicand to the product
    rjmp m_skip     ; if 0: jump over the add
    add r18, r20
    adc r19, r21
```

```
m_skip:
    lsl r20         ; multiplicand <<= 1
    rol r21
    lsr r17         ; next multiplier bit down into position 0
    dec r22
    brne m_loop

    sts product, r19    ; MSB
    sts product+1, r18  ; LSB
    ret

copybytes:
    lpm r0, z+
    st x+, r0
    dec r16
    brne copybytes
    ret

fin:
    rjmp fin

pmv1:
    .db 0x0D, 0x0B      ; 13 * 11 = 143 = 0x008F

    .dseg
    value1: .byte 1
    value2: .byte 1
    product: .byte 2
```